ӨГӨГДЛИЙН САНГИЙН УДИРДЛАГА

F.ITM237

ЛАБОРАТОРИЙН УДИРДАМЖ №3

Дэвшилтэт SQL: JOIN, View, Index, Transaction

Ажиллах орчин: Microsoft SQL Server | **Нийт оноо: 3 оноо**

Мэдээллийн технологийн тэнхим | С.Өлзийбаяр (Ph.D)

1 ЛАБОРАТОРИЙН ЗОРИЛГО

Энэхүү лабораторийн ажил нь 4-р долоо хоногийн лекцийн агуулга болох дэвшилтэт SQL функцуудыг практикт хэрэглэх ур чадварыг эзэмшүүлэхэд чиглэнэ. Оюутнууд дараах чадваруудыг эзэмшинэ:

- JOIN операторуудыг (INNER JOIN, LEFT JOIN) ашиглан олон хүснэгтийн өгөгдлийг нэгтгэх
- Subquery ашиглан нарийн нөхцөлт асуулга бичих
- VIEW үүсгэж, өгөгдлийн хандалтыг хялбарчлах
- INDEX ашиглан асуулгын гүйцэтгэлийг сайжруулах
- TRANSACTION удирдлагыг (COMMIT, ROLLBACK, SAVEPOINT) зөв хэрэглэх
- ACID шинж чанарыг ойлгож, зэрэгцээ гүйлгээний асуудлуудыг таних

2 ШААРДЛАГАТАЙ ПРОГРАМ ХАНГАМЖ

- Microsoft SQL Server 2019 эсвэл түүнээс дээш хувилбар
- SQL Server Management Studio (SSMS) эсвэл Azure Data Studio
- Lab02-оос үүсгэсэн UniversityDB өгөгдлийн сан (Department, Student хүснэгтүүд)

3 ОНОЛЫН ҮЙЛ

3.1 JOIN операторууд

JOIN нь хоёр ба түүнээс дээш хүснэгтийн өгөгдлийг нийтлэг багана (ихэвчлэн гадаад түлхүүр) дээр үндэслэн нэгтгэх SQL арга юм.

Команд / Хязгаарлалт	Тайлбар
`INNER JOIN`	Хоёр хүснэгтэд таарсан мөрүүдийг буцаана. Таарахгүй мөр ороогүй байна
`LEFT JOIN`	Зүүн хүснэгтийн бүх мөр + баруун хүснэгтэд тохирсон мөр. Тохирогдоогүй бол NULL

`RIGHT JOIN`	Баруун хүснэгтийн бүх мөр + зүүн хүснэгтэд тохирсон мөр. Тохирогдоогүй бол NULL
`FULL OUTER JOIN`	Хоёр хүснэгтийн бүх мөрийг буцааж, тохирсон газар нэгтгэнэ
`CROSS JOIN`	Хоёр хүснэгтийн бүх мөрийн Декартын үржвэрийг буцаана

3.2 Subquery (Дэд асуулга)

Дэд асуулга нь нэг SQL асуулгын дотор бичигдсэн өөр нэг асуулга бөгөөд үндсэн асуулгадаа ашиглах утгыг гаргаж өгдөг.

Команд / Хязгаарлалт	Тайлбар
`Нэг утгат`	Нэг утга буцааж, = , > , < операторуудтай хэрэглэнэ
`Олон утгат`	Хэд хэдэн утга буцааж, IN, ANY, ALL операторуудтай хэрэглэнэ
`Correlated`	Гадаад асуулгатай холбоотой, мөр бүрд давтан ажиллана

3.3 View, Index, Sequence

Команд / Хязгаарлалт	Тайлбар
`VIEW`	Виртуал хүснэгт. SQL асуулгыг хадгалж, хялбаршуулсан хандалт олгоно
`INDEX`	Өгөгдлийн хайлтыг хурдасгах бүтцийн элемент. Хайлтын зардлыг бууруулна
`SEQUENCE`	Автоматаар өөрчлөгдөх дугаар үүсгэх механизм (Oracle). MSSQL: IDENTITY

3.4 Transaction ба ACID

Гүйлгээ (Transaction) нь SQL өгөгдлийн санд хийж буй нэг буюу олон үйлдлийг нэгдмэл нэгж болгон гүйцэтгэх механизм юм.

Команд / Хязгаарлалт	Тайлбар
`Atomicity (A)`	Гүйлгээний бүх үйлдэл бүрэн хийгдэх эсвэл огт хийгдэхгүй — хагас байдал байхгүй
`Consistency (C)`	Гүйлгээний өмнө ба дараа өгөгдлийн сан зөв ба тодорхой төлөвт байх ёстой
`Isolation (I)`	Нэгэн зэрэг явагдах гүйлгээнүүд бие биедээ нөлөөлж болохгүй
`Durability (D)`	COMMIT хийсний дараа хийгдсэн өөрчлөлт системийн доголдолд ч хадгалагдана

0 ӨГӨГДЛИЙН БЭЛТГЭЛ (Лаб эхлэхээс өмнө)

Lab02-оос үүсгэсэн UniversityDB өгөгдлийн санд холбогдоно. Хэрэв Department болон Student хүснэгтүүд байхгүй бол дараах кодоор үүсгэнэ:

```
USE UniversityDB;
GO

-- Department хүснэгт (Lab02-оос аль хэдийн байгаа бол алгасна)
-- Student хүснэгт дэх мэдээллийг шалгах
SELECT COUNT(*) AS DepartmentCount FROM Department;
SELECT COUNT(*) AS StudentCount FROM Student;

-- Туршилтын нэмэлт өгөгдөл оруулах
INSERT INTO Student (FirstName, LastName, RegistrationNumber, Email,
    DateOfBirth, Gender, DepartmentID, EnrollmentDate, Status, GPA)
VALUES
    (N'Мөнхбат', N'Гантулга', 'B230101', 'munkh@student.edu.mn',
     '2005-06-10', N'Эрэгтэй', 2, '2023-09-01', N'Идэвхтэй', 3.10),
    (N'Цэцэгмаа', N'Лхагва', 'B220099', 'tsetseg@student.edu.mn',
     '2004-02-28', N'Эмэгтэй', NULL, '2022-09-01', N'Идэвхтэй', 2.80);
```

4 ПРАКТИК ДААЛГАВАР

Даалгавар 1: JOIN операторууд — 1.0 оноо

1а. INNER JOIN — Оюутан ба тэнхим нэгтгэх (0.3 оноо)

Бүх оюутны нэр, регистрийн дугаар болон тэдний тэнхимийн нэрийг нэг асуулгаар харуул.

```
-- INNER JOIN ашиглан оюутан ба тэнхимийн мэдээлэл нэгтгэх
SELECT
    s.FirstName + ' ' + s.LastName   AS ОюутанНэр,
    s.RegistrationNumber              AS РегистрДугаар,
    d.DepartmentName                  AS ТэнхимНэр,
    s.GPA,
    s.Status
FROM Student s
INNER JOIN Department d
    ON s.DepartmentID = d.DepartmentID;
```

> ⚠ **Анхаарах:**
> INNER JOIN нь хоёр хүснэгтэд тохирсон мөрүүдийг л буцаана. Тэнхимгүй оюутнууд (DepartmentID = NULL) үр дүнд ороогүй байна. Хэдэн оюутан дутуу байгааг тоол.

1б. LEFT JOIN — Тэнхимгүй оюутнуудыг илрүүлэх (0.3 оноо)

Бүх оюутнуудыг харуулж, тэнхимд бүртгэлгүй оюутнуудыг тусгайлан гаргаж ирэх хоёр асуулга бич.

```
-- LEFT JOIN: Бүх оюутан + тохирсон тэнхим
SELECT
    s.FirstName + ' ' + s.LastName   AS ОюутанНэр,
    s.RegistrationNumber,
    ISNULL(d.DepartmentName, N'Тэнхимгүй') AS ТэнхимНэр
FROM Student s
LEFT JOIN Department d
    ON s.DepartmentID = d.DepartmentID;

-- Зөвхөн тэнхимгүй оюутнуудыг шүүж гаргах
SELECT
```

```
    s.FirstName + ' ' + s.LastName   AS ОюутанНэр,
    s.RegistrationNumber
FROM Student s
LEFT JOIN Department d
    ON s.DepartmentID = d.DepartmentID
WHERE d.DepartmentID IS NULL;
```

1в. Тэнхим бүрийн оюутны тоо — GROUP BY + JOIN (0.4 оноо)

Тэнхим бүрт хэдэн оюутан байгааг, тэнхимийн нэрийг харуулан тоолж, оюутны тоогоор буурах эрэмбээр гарга.

```
-- Тэнхим бүрийн оюутны тоо
SELECT
    d.DepartmentName                AS ТэнхимНэр,
    COUNT(s.StudentID)              AS ОюутныТоо,
    AVG(s.GPA)                      AS ДундажGPA,
    MAX(s.GPA)                      AS ХамгийнӨндөрGPA
FROM Department d
LEFT JOIN Student s
    ON d.DepartmentID = s.DepartmentID
GROUP BY d.DepartmentID, d.DepartmentName
ORDER BY ОюутныТоо DESC;
```

Даалгавар 2: Subquery, View ба Index — 1.0 оноо

2а. Subquery — Дундаж GPA-аас дээш оюутнууд (0.3 оноо)

Дэд асуулга ашиглан бүх оюутны дундаж GPA-аас өндөр GPA-тай идэвхтэй оюутнуудыг гаргаж ирэх асуулга бич.

```
-- Дэд асуулга: дундаж GPA-аас дээш оюутнууд
SELECT
    s.FirstName + ' ' + s.LastName   AS ОюутанНэр,
    s.GPA,
    d.DepartmentName                  AS Тэнхим
FROM Student s
INNER JOIN Department d ON s.DepartmentID = d.DepartmentID
WHERE s.GPA > (SELECT AVG(GPA) FROM Student WHERE Status = N'Идэвхтэй')
  AND s.Status = N'Идэвхтэй'
ORDER BY s.GPA DESC;

-- Хяналтын асуулга: дундаж GPA-г шалгах
SELECT AVG(GPA) AS ДундажGPA FROM Student WHERE Status = N'Идэвхтэй';
```

2б. View үүсгэх ба ашиглах (0.4 оноо)

Идэвхтэй оюутнуудын нэр, тэнхим, GPA-г нэгтгэсэн VIEW үүсгэ. Дараа нь тухайн VIEW-аас JOIN болон WHERE ашиглан шүүлт хийж харуул.

```
-- View үүсгэх
CREATE VIEW vw_ActiveStudents AS
SELECT
    s.StudentID,
    s.FirstName + ' ' + s.LastName   AS ПүрэнНэр,
    s.RegistrationNumber,
    s.GPA,
    s.EnrollmentDate,
    d.DepartmentName,
    d.Building
FROM Student s
```

```
INNER JOIN Department d ON s.DepartmentID = d.DepartmentID
WHERE s.Status = N'Идэвхтэй';
GO

-- View-аас асуулга хийх
SELECT * FROM vw_ActiveStudents;

-- View-аас тэнхимээр шүүж харах
SELECT ПүрэнНэр, GPA, DepartmentName
FROM vw_ActiveStudents
WHERE DepartmentName = N'Мэдээллийн технологи'
ORDER BY GPA DESC;
```

2в. Index үүсгэх (0.3 оноо)

Оюутны хүснэгтэд хайлтыг хурдасгах зорилгоор дараах гурван индексийг үүсгэ.

```
-- 1. GPA баганад энгийн индекс
CREATE INDEX IDX_Student_GPA
    ON Student(GPA);

-- 2. Status баганад индекс
CREATE INDEX IDX_Student_Status
    ON Student(Status);

-- 3. Тэнхим + GPA нийлмэл индекс (хамгийн их ашиглагддаг хайлтад зориулав)
CREATE INDEX IDX_Student_Dept_GPA
    ON Student(DepartmentID, GPA DESC);

-- Үүсгэсэн индексүүдийг шалгах
SELECT name, type_desc, is_unique
FROM sys.indexes
WHERE object_id = OBJECT_ID('Student');
```

Даалгавар 3: Transaction ба Concurrency Control — 1.0 оноо

3а. Банкны шилжүүлгийн Transaction (0.4 оноо)

Дараах хоёр хүснэгтийг үүсгэж, банкны мөнгөн шилжүүлгийг TRANSACTION дотор хийгэх бөгөөд алдаа гарвал ROLLBACK хийх кодыг бич.

```
-- Туршилтын хүснэгт үүсгэх
CREATE TABLE Accounts (
    AccountNumber INT PRIMARY KEY,
    OwnerName     NVARCHAR(100) NOT NULL,
    Balance       DECIMAL(12,2) CHECK (Balance >= 0)
);

INSERT INTO Accounts VALUES
    (1001, N'Бат',    1000000),
    (1002, N'Сарнай',  500000);

-- ====================================
-- TRANSACTION: мөнгөн шилжүүлэг
-- ====================================
BEGIN TRANSACTION;

BEGIN TRY
    -- 1-р алхам: 1001 дансаас 200,000 хасах
    UPDATE Accounts
    SET Balance = Balance - 200000
    WHERE AccountNumber = 1001;
```

```
    -- 2-р алхам: 1002 данс руу 200,000 нэмэх
    UPDATE Accounts
    SET Balance = Balance + 200000
    WHERE AccountNumber = 1002;

    COMMIT;
    PRINT N'Шилжүүлэг амжилттай гүйцэтгэгдлээ.';
END TRY
BEGIN CATCH
    ROLLBACK;
    PRINT N'Алдаа гарлаа. Шилжүүлэг цуцлагдлаа: ' + ERROR_MESSAGE();
END CATCH;

-- Үр дүнг шалгах
SELECT * FROM Accounts;
```

> **Туршилтын санал:**
> Дараах тохиолдолд яах вэ гэдгийг туршиж үз: Balance < 0 болохуйц их мөнгө хасахыг оролдоорой (жишээ нь: 2,000,000). CHECK хязгаарлалт ажиллаж, ROLLBACK хийгдэх ёстой.

3б. SAVEPOINT ашиглах (0.3 оноо)

SAVEPOINT ашиглан гүйлгээний зарим хэсгийг л цуцлах боломжийг харуулах жишээ бич.

```
BEGIN TRANSACTION;

    -- 1-р үйлдэл: 1001 дансны мэдээлэл шинэчлэх
    UPDATE Accounts
    SET Balance = Balance - 50000
    WHERE AccountNumber = 1001;

    PRINT N'1-р үйлдэл хийгдлээ. Savepoint тавих...';
    SAVE TRANSACTION SavePoint1;

    -- 2-р үйлдэл: 1002 дансны мэдээлэл шинэчлэх
    UPDATE Accounts
    SET Balance = Balance - 999999  -- Санаатайгаар буруу мөнгө хасаж байна
    WHERE AccountNumber = 1002;

    -- 2-р үйлдлийг цуцлаж Savepoint руу буцах
    ROLLBACK TRANSACTION SavePoint1;
    PRINT N'2-р үйлдэл цуцлагдаж, SavePoint1 руу буцлаа.';

-- Зөвхөн 1-р үйлдлийг хадгалах
COMMIT;
PRINT N'Зөвхөн 1-р үйлдэл хадгалагдлаа.';

SELECT * FROM Accounts;
```

3в. Isolation Level тохируулах (0.3 оноо)

Дараах кодыг ажиллуулж, SERIALIZABLE isolation level-ийн тухай ойлголтоо харуул. Дараа нь доорх асуултад хариулна уу.

```
-- Isolation level тохируулах
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

BEGIN TRANSACTION;
    SELECT AccountNumber, OwnerName, Balance
```

```
    FROM Accounts
    WHERE AccountNumber = 1001;

    -- Данс уншсаны дараа шинэчлэх
    UPDATE Accounts
    SET Balance = Balance - 30000
    WHERE AccountNumber = 1001;
COMMIT;

-- Isolation level-ийг буцаах (анхдагч)
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

Бичгийн асуулт (3в-ийн хариулт):
Дараах тайлбарыг өөрийн үгээр 3-5 өгүүлбэрээр бич: • READ COMMITTED ба SERIALIZABLE isolation level-ийн ялгаа юу вэ? • Аль нь аюулгүй вэ? Аль нь хурдан вэ? • Dirty Read, Phantom Read гэж юу вэ? Жишээ оруул.

5 ҮНЭЛГЭЭНИЙ РУБРИК

Лабораторийн ажлыг дараах шалгуурын дагуу нийт 3.0 оноогоор үнэлнэ:

№	Даалгавар	Оноо	Үнэлгээний шалгуур
1	**Өгөгдлийн бэлтгэл**	0.0	Хүснэгтүүд зөв үүсгэгдсэн, өгөгдөл оруулагдсан
2	**JOIN асуулгууд**	1.0	INNER, LEFT JOIN зөв ажиллаж, үр дүн зөв
3	**View, Index, Subquery**	1.0	View үүсгэж хэрэглэсэн, Index зөв, Subquery ажиллаж байгаа
4	**Transaction**	1.0	COMMIT, ROLLBACK, SAVEPOINT зөв ашигласан, ACID тайлбарласан
НИЙТ ОНОО:		**3.0**	

Нэмэлт үнэлгээний шалгуур

- Кодын форматчлал: SQL код уншигдахуйц, indent зөв, монгол тайлбар бичсэн
- Алдааны шалгалт: Хязгаарлалтуудыг зориуд зөрчиж туршсан (эерэг ба сөрөг тест)
- Бүрэн гүйцэтгэл: Бүх sub-даалгавар амжилттай дуусгасан
- Багшид.sql файл дээр туршиж үзүүлж хамгаалсан

6 ХУГАЦАА БА ДҮРЭМ

- Лабораторийн ажлыг 5-р долоо хоногт багтан хамгаалах
- Хугацаа хэтэрсэн долоо хоног бүрт 1 оноог бууруулж тооцно
- Бусдын ажлыг хуулбарласан тохиолдолд 0 оноо авна

Амжилт хүсье!
Асуулт гарвал багш эсвэл туслах багштайгаа холбогдоно уу.